

SE446: Big Data Engineering

Week 8A: Spark on the Cluster — Deployment & Performance Tuning

Prof. Anis Koubaa

SE 446

Alfaisal University

https://github.com/aniskoubaa/big_data_course

Spring 2026

Today's Agenda

- 1 Recap & The Spark Journey
- 2 Deploying Jobs with `spark-submit`
- 3 Spark Web UI Deep Dive
- 4 Caching & Persistence
- 5 Partition Tuning
- 6 Broadcast Joins
- 7 Summary

Outline

- 1 Recap & The Spark Journey
- 2 Deploying Jobs with `spark-submit`
- 3 Spark Web UI Deep Dive
- 4 Caching & Persistence
- 5 Partition Tuning
- 6 Broadcast Joins
- 7 Summary

Learning Objectives

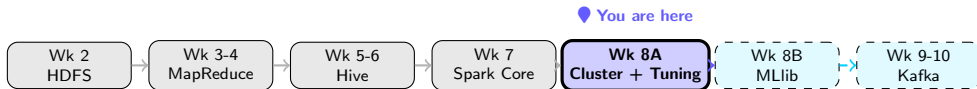
By the end of this session you will be able to:

- 1 Submit Spark jobs to a YARN cluster using `spark-submit`
- 2 Distinguish `client` vs `cluster` deploy modes
- 3 Read the Spark Web UI: Jobs, Stages, Tasks, Storage, Executors
- 4 Apply caching with `.cache()` and `.persist()`
- 5 Tune partition count for optimal parallelism
- 6 Use broadcast joins for small lookup tables

Dataset

Chicago Crimes (HDFS) — same dataset from Week 7

The Big Data Journey — Where Are We?



What you already know

- RDDs, DataFrames, Spark SQL
- PySpark in **interactive shell**
- Ran jobs locally or on the shell



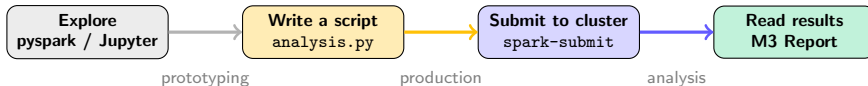
What changes today

- Submit **real jobs** to the cluster
- Read the **Spark Web UI**
- Make jobs **fast** (tuning)
- **M3 milestone** uses these skills

Outline

- 1 Recap & The Spark Journey
- 2 Deploying Jobs with `spark-submit`
- 3 Spark Web UI Deep Dive
- 4 Caching & Persistence
- 5 Partition Tuning
- 6 Broadcast Joins
- 7 Summary

From Exploration to Submission — Your M3 Workflow



>_ PySpark Shell / Jupyter

- Type code, see results instantly
- Great for **exploring** the dataset
- Driver runs on **your machine**
- Cannot run unattended

spark-submit on YARN

- Runs a `.py` script as a batch job
- Works on the **cluster's resources**
- Reproducible — versioned in GitHub
- Required for **M3 submission**

The Rule

Prototype in Jupyter → polish to `.py` → `spark-submit` → submit results for M3.

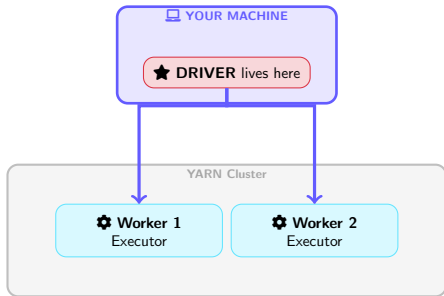
spark-submit Syntax

```
spark-submit \  
  --master yarn \  
  --deploy-mode client \  
  --num-executors 2 \  
  --executor-memory 1g \  
  --executor-cores 2 \  
  --driver-memory 512m \  
  --conf spark.sql.shuffle.partitions=4 \  
  crime_analysis.py
```

Flag	Purpose
--master yarn	Run on YARN cluster
--deploy-mode	Where Driver runs (client/cluster)
--num-executors	Number of executor JVMs
--executor-memory	RAM per executor
--executor-cores	CPU cores per executor
--conf	Custom Spark configuration

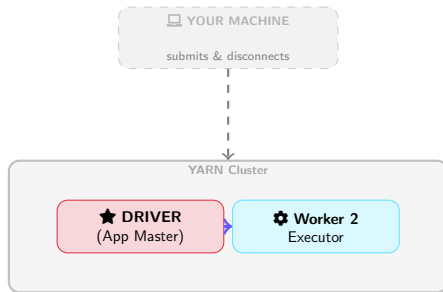
Deploy Mode: client vs cluster

Client Mode



- ✓ See `print()` output live
- ✓ Great for debugging
- ✗ SSH drops = job dies

Cluster Mode



- ✓ Survives SSH disconnect
- ✓ Best for production
- ✗ No live output (check YARN logs)

Deploy Modes — Think of It Like a Phone Call



Client Mode = Speakerphone

- You're on the call the **entire time**
- You hear everything live (`print()` output)
- If you **hang up** (SSH drops), the call ends
- Best for: **development & debugging**



Cluster Mode = Voicemail

- You leave a message and **hang up**
- The work happens **without you**
- Check results later via YARN logs
- Best for: **production & long jobs**



Common Pitfall

Students often use cluster mode during development and wonder why they see no output. **Use client mode first**, switch to cluster for final submission.

Quick Check: You're debugging and need to see `print()` output. Which mode?

Deploy Modes — Think of It Like a Phone Call



Client Mode = Speakerphone

- You're on the call the **entire time**
- You hear everything live (`print()` output)
- If you **hang up** (SSH drops), the call ends
- Best for: **development & debugging**



Cluster Mode = Voicemail

- You leave a message and **hang up**
- The work happens **without you**
- Check results later via YARN logs
- Best for: **production & long jobs**



Common Pitfall

Students often use cluster mode during development and wonder why they see no output. **Use client mode first**, switch to cluster for final submission.

Quick Check: You're debugging and need to see `print()` output. Which mode?

Client mode!

Setting Up Spark Client on Your Computer

1. Install Spark & Set Environment

Download the **same Spark version** as your cluster from `spark.apache.org`, then:

```
export SPARK_HOME=/opt/spark           # where you extracted Spark
export PATH=$SPARK_HOME/bin:$PATH
export HADOOP_CONF_DIR=/etc/hadoop/conf
```

2. Point to Your Cluster

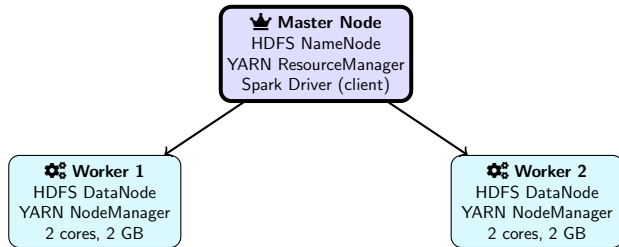
Copy these three files from the cluster into `$HADOOP_CONF_DIR`: `core-site.xml`, `hdfs-site.xml`, `yarn-site.xml`. Your machine must be able to reach the cluster nodes over the network.

3. Submit in Client Mode

```
spark-submit --master yarn --deploy-mode client \
  --num-executors 2 --executor-memory 1g my_analysis.py
```

Your laptop is the **Driver** → `print()` output appears in your terminal.

Our Cluster Configuration



Spark Web UI: <http://master:4040> — YARN UI: <http://master:8088>

Cluster Resources

Total: 4 cores, 4 GB RAM

Request $\leq$ 1g per executor

Analogy

Master = **project manager** (coordinates)

Workers = **team members** (do the work)

Understanding the Cluster — Resource Math



Master = Coordinator

- **NameNode**: knows where data blocks are
- **ResourceManager**: allocates containers
- **Driver** (client): plans execution



Workers = Muscle

- **DataNode**: stores data blocks
- **NodeManager**: manages containers
- **Executors**: run your tasks



Resource Math

Total cluster: 4 cores, 4 GB. With `--num-executors 2 --executor-cores 2 --executor-memory 1g`, you use **all 4 cores** and **2 GB** (leaving headroom for OS and YARN overhead).

Example Script: Crime Counts

```
# crime_counts.py
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, count

spark = SparkSession.builder \
    .appName("CrimeCounts") \
    .getOrCreate()

df = spark.read.csv("hdfs:///data/chicago_crimes.csv",
                    header=True, inferSchema=True)

result = df.groupBy("Primary Type") \
    .agg(count("*").alias("cnt")) \
    .orderBy(col("cnt").desc())

result.show(10)
spark.stop()
```

Submit to YARN

```
spark-submit --master yarn --deploy-mode client crime_counts.py
```

You have a script that takes 30 minutes to run.

Your laptop might lose WiFi during the run.

Which deploy mode should you use?

What flag do you add to spark-submit?

Discuss with your neighbor for 30 seconds.

You have a script that takes 30 minutes to run.

Your laptop might lose WiFi during the run.

Which deploy mode should you use?

What flag do you add to spark-submit?

Discuss with your neighbor for 30 seconds.

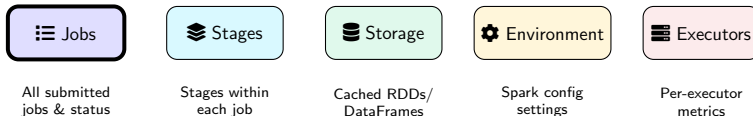
✓ Answer

`--deploy-mode cluster` — the Driver runs inside YARN, so even if your WiFi drops, the job continues. Retrieve output with `yarn logs -applicationId <id>`.

Outline

- 1 Recap & The Spark Journey
- 2 Deploying Jobs with `spark-submit`
- 3 Spark Web UI Deep Dive
- 4 Caching & Persistence
- 5 Partition Tuning
- 6 Broadcast Joins
- 7 Summary

Spark Web UI — Overview



While App Runs

hdfs.aniskoubaa.org/spark-ui/

After App Finishes

hdfs.aniskoubaa.org/spark-history/

Key Principle

Each **action** creates a **job**. Each job has one or more **stages** (separated by shuffles). Each stage has multiple **tasks** (one per partition).

Predict the Execution Plan

```
df.groupBy("Primary Type").count().show()
```

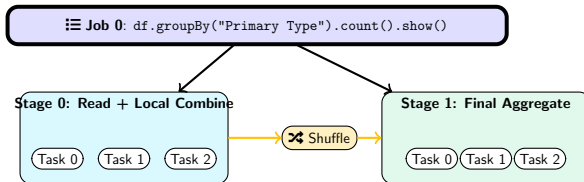
Before I show you the answer: How many **jobs**? How many **stages**? Why?

Hint: Which operation forces a shuffle?

Predict the Execution Plan

```
df.groupBy("Primary Type").count().show()
```

Before I show you the answer: How many **jobs**? How many **stages**? Why?
Hint: Which operation forces a shuffle?



✓ Answer

1 job (one action: `.show()`), **2 stages** (separated by the `groupBy` shuffle).

Understanding Jobs, Stages, and Tasks

Jobs — Triggered by Actions

- Every **action** (`.show()`, `.count()`, `.collect()`, `.write`) creates a new **Job**
- Transformations (`.filter()`, `.groupBy()`) are lazy — they do **not** trigger Jobs

Stages — Separated by Shuffles

- A Job is split into **Stages** at every **shuffle boundary** (`groupBy`, `join`)
- Fewer stages = fewer shuffles = **better performance**

Tasks — One Per Partition

- Each stage launches **one task per partition** (4 partitions = 4 tasks)
- Tasks run in **parallel** across executor cores
- A slow task (straggler) slows the **entire stage** — check the Tasks tab!

Spotting Performance Problems in the Web UI

Symptom	Where to Look	Cause & Fix
One task 10× slower	Tasks tab — Duration	Data skew: one partition too large. Fix: <code>repartition()</code> or salting.
High GC Time	Executors tab	Memory pressure: increase <code>executor-memory</code> or reduce partition size.
Large shuffle spill	Stages tab — Spill column	Not enough RAM: data spills to disk. Fix: more memory or fewer partitions.
Many empty tasks	Stages tab — Task count	Over-partitioned: too many partitions. Fix: <code>coalesce()</code> .

Lab Task

In today's lab, you'll observe each of these in the real Spark Web UI while running experiments on the cluster.

Live Demo: Spark History Server



Event log directory: hdfs://spark-logs

Last updated: 2026-04-01 08:32:55

Client local time zone: Asia/Riyadh

Search:

Version	App ID	App Name	Started	Completed	Duration	Spark User	Last Updated	Event Log
3.5.4	application_1771402826595_0268	SE446-MyJob	2026-04-01 08:30:29	2026-04-01 08:31:31	1.0 min	root	2026-04-01 08:31:32	Download
3.5.4	application_1771402826595_0267	SE446-QuickStart-CrimeAnalysis	2026-04-01 08:28:39	2026-04-01 08:29:53	1.2 min	root	2026-04-01 08:29:53	Download
3.5.4	application_1771402826595_0075	HDFS-Read-Test	2026-03-02 08:09:49	2026-03-02 08:10:33	44 s	root	2026-03-02 08:10:34	Download
3.5.4	application_1771402826595_0074	PySpark-Test	2026-03-02 08:08:36	2026-03-02 08:09:15	39 s	root	2026-03-02 08:09:15	Download
3.5.4	application_1771402826595_0073	PySpark-Test	2026-03-02 08:04:42	2026-03-02 08:05:22	39 s	root	2026-03-02 08:05:22	Download
3.5.4	application_1771402826595_0072	PySpark-Test	2026-03-02 08:03:46	2026-03-02 08:04:25	39 s	root	2026-03-02 08:04:25	Download
3.5.4	application_1771402826595_0070	Spark PI	2026-03-02 07:51:45	2026-03-02 07:52:22	37 s	root	2026-03-02 07:52:22	Download

Showing 1 to 7 of 7 entries

[Show incomplete applications](#)

What you see: All completed apps with ID, name, duration. Click an **App ID** to drill into jobs/stages/tasks.
hdfs.aniskoubaa.org/spark-history/



Live Demo: Jobs Tab

Spark

2.6.4

Jobs

StagesStorageEnvironmentExecutorsSQL / DataFrame

SE446-QuickStart-CrimeAnalysis application UI

Spark Jobs ^(?)

User: root

Total Uptime: 5.2 min

Scheduling Mode: FIFO

Completed Jobs: 12

[Event Timeline](#)

[Completed Jobs \(12\)](#)

Page: 1

1 Pages. Jump to: 1. Show: 100 Items in a page. Go

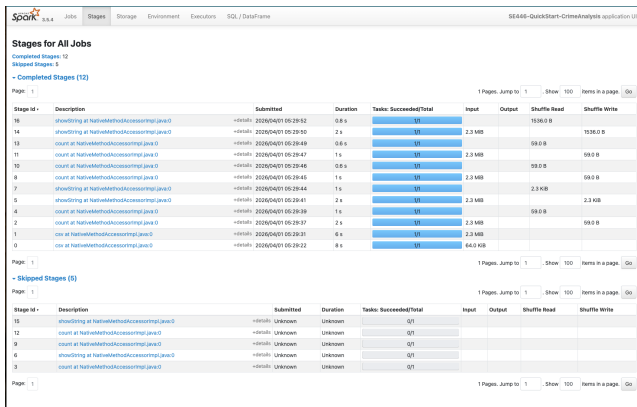
Job id	Description	Submitted	Duration	Stages: Succeeded/Total	Tasks (for all stages): Succeeded/Total
11	showString at NativeMethodAccessorImpl.java:0 showString at NativeMethodAccessorImpl.java:0	2026/04/01 05:29:52	0.0 s	1/1 (1 skipped)	1/1 (1 skipped)
10	showString at NativeMethodAccessorImpl.java:0 showString at NativeMethodAccessorImpl.java:0	2026/04/01 05:29:50	2 s	1/1	1/1
9	count at NativeMethodAccessorImpl.java:0 count at NativeMethodAccessorImpl.java:0	2026/04/01 05:29:49	0.0 s	1/1 (1 skipped)	1/1 (1 skipped)
8	count at NativeMethodAccessorImpl.java:0 count at NativeMethodAccessorImpl.java:0	2026/04/01 05:29:47	1 s	1/1	1/1
7	count at NativeMethodAccessorImpl.java:0 count at NativeMethodAccessorImpl.java:0	2026/04/01 05:29:46	0.0 s	1/1 (1 skipped)	1/1 (1 skipped)
6	count at NativeMethodAccessorImpl.java:0 count at NativeMethodAccessorImpl.java:0	2026/04/01 05:29:45	1 s	1/1	1/1
5	showString at NativeMethodAccessorImpl.java:0 showString at NativeMethodAccessorImpl.java:0	2026/04/01 05:29:44	1 s	1/1 (1 skipped)	1/1 (1 skipped)
4	showString at NativeMethodAccessorImpl.java:0 showString at NativeMethodAccessorImpl.java:0	2026/04/01 05:29:41	3 s	1/1	1/1
3	count at NativeMethodAccessorImpl.java:0 count at NativeMethodAccessorImpl.java:0	2026/04/01 05:29:39	1 s	1/1 (1 skipped)	1/1 (1 skipped)
2	count at NativeMethodAccessorImpl.java:0 count at NativeMethodAccessorImpl.java:0	2026/04/01 05:29:37	2 s	1/1	1/1
1	csv at NativeMethodAccessorImpl.java:0 csv at NativeMethodAccessorImpl.java:0	2026/04/01 05:29:31	6 s	1/1	1/1
0	csv at NativeMethodAccessorImpl.java:0 csv at NativeMethodAccessorImpl.java:0	2026/04/01 05:29:22	8 s	1/1	1/1

Page: 1

1 Pages. Jump to: 1. Show: 100 Items in a page. Go

What you see: One row per action (`.count()`, `.show()`). **Stages Succeeded/Total** — skipped = cached data reused. **Tasks bar (blue)** — stalled bar = straggler task.

Live Demo: Stages Tab



The screenshot shows the Spark Stages Tab for a job named 'SE446-QuickStart-CrimeAnalysis application UI'. It displays two sections: 'Completed Stages (12)' and 'Skipped Stages (0)'. Each section contains a table with columns for Stage ID, Description, Submitted, Duration, Tasks: Succeeded/Total, Input, Output, Shuffle Read, and Shuffle Write. The completed stages show various tasks like 'showString at NativeMethodAccessorImpl.java:0' and 'count at NativeMethodAccessorImpl.java:0' with their respective durations and shuffle data. The skipped stages are listed as 'Unknown'.

Stage id	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
16	showString at NativeMethodAccessorImpl.java:0	2026/04/01 05:28:52	0.8 s	1/1			1536.0 B	
14	showString at NativeMethodAccessorImpl.java:0	2026/04/01 05:28:50	2 s	1/1	2.3 MB			1536.0 B
13	count at NativeMethodAccessorImpl.java:0	2026/04/01 05:28:49	0.6 s	1/1			59.0 B	
11	count at NativeMethodAccessorImpl.java:0	2026/04/01 05:28:47	1 s	1/1	2.3 MB			59.0 B
10	count at NativeMethodAccessorImpl.java:0	2026/04/01 05:28:46	0.6 s	1/1			59.0 B	
8	count at NativeMethodAccessorImpl.java:0	2026/04/01 05:28:45	1 s	1/1	2.3 MB			59.0 B
7	showString at NativeMethodAccessorImpl.java:0	2026/04/01 05:28:44	1 s	1/1		2.3 KB		
5	showString at NativeMethodAccessorImpl.java:0	2026/04/01 05:28:41	2 s	1/1	2.3 MB			2.3 KB
4	count at NativeMethodAccessorImpl.java:0	2026/04/01 05:28:39	1 s	1/1			59.0 B	
2	count at NativeMethodAccessorImpl.java:0	2026/04/01 05:28:37	2 s	1/1	2.3 MB			59.0 B
1	csv at NativeMethodAccessorImpl.java:0	2026/04/01 05:28:31	6 s	1/1	2.3 MB			
0	csv at NativeMethodAccessorImpl.java:0	2026/04/01 05:28:22	8 s	1/1	64.0 KB			

Stage id	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
15	showString at NativeMethodAccessorImpl.java:0	Unknown	Unknown	0/1				
12	count at NativeMethodAccessorImpl.java:0	Unknown	Unknown	0/1				
9	count at NativeMethodAccessorImpl.java:0	Unknown	Unknown	0/1				
6	showString at NativeMethodAccessorImpl.java:0	Unknown	Unknown	0/1				
3	count at NativeMethodAccessorImpl.java:0	Unknown	Unknown	0/1				

What you see: Completed vs Skipped (skipped = cached). **Shuffle Read/Write** — where bottlenecks hide. Large shuffle write + small read = **data skew**.

Live Demo: Executors Tab

 3.5.4

JobsStagesStorageEnvironmentExecutorsSQL / DataFrame

SE446-QuickStart-CrimeAnalysis application UI

Executors

[Show Additional Metrics](#)

Summary

	#	RDD Blocks	Storage Memory	Disk Used	Cores	Active Tasks	Failed Tasks	Complete Tasks	Total Tasks	Task Time (GC Time)	Input	Shuffle Read	Shuffle Write	Excluded
Active(2)	0	0.0 B / 392.7 MB	0.0 B	1	0	0	12	12	1.7 min (0.3 s)	13.7 MB	4 KB	4 KB	0	
Dead(0)	0	0.0 B / 0.0 B	0.0 B	0	0	0	0	0	0.0 ms (0.0 ms)	0.0 B	0.0 B	0.0 B	0	
Total(2)	0	0.0 B / 392.7 MB	0.0 B	1	0	0	12	12	1.7 min (0.3 s)	13.7 MB	4 KB	4 KB	0	

Executors

Show: 20 entries

Search:

Executor ID	Address	Status	RDD Blocks	Storage Memory	Disk Used	Cores	Active Tasks	Failed Tasks	Complete Tasks	Total Tasks	Task Time (GC Time)	Input	Shuffle Read	Shuffle Write	Logs	Add Time	Remove Time
driver	master-node-39717	Active	0	0.0 B / 127.2 MB	0.0 B	0	0	0	0	0	1.2 min (0.0 ms)	0.0 B	0.0 B	0.0 B		2026-04-01 08:28:56	-
1	worker-node-1-36513	Active	0	0.0 B / 285.5 MB	0.0 B	1	0	0	12	12	25 s (0.3 s)	13.7 MB	4 KB	4 KB	stdout stderr	2026-04-01 08:28:10	-

Showing 1 to 2 of 2 entries

Previous 1 Next

Miscellaneous Process

Show: 30 entries

Search:

Process ID	Address	Status	Cores	Logs
yarn-arn	worker-node-2-8042	Active	1	stdout stderr

Showing 1 to 1 of 1 entries

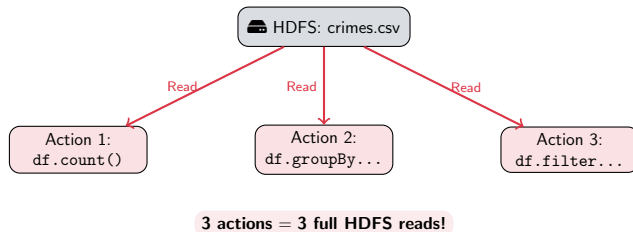
Previous 1 Next

What you see: Per-executor metrics — tasks, storage memory, shuffle I/O. **GC Time** high = memory pressure. Our cluster: 1 driver (master) + 1 executor (worker-node-1).

Outline

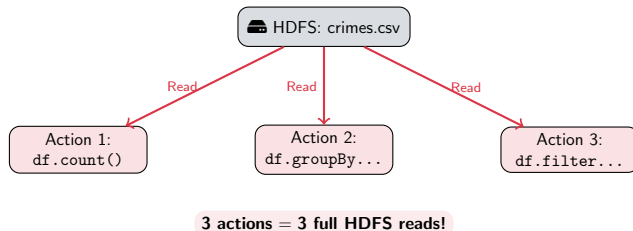
- 1 Recap & The Spark Journey
- 2 Deploying Jobs with `spark-submit`
- 3 Spark Web UI Deep Dive
- 4 Caching & Persistence**
- 5 Partition Tuning
- 6 Broadcast Joins
- 7 Summary

The Problem: Repeated HDFS Reads



Quiz: You run 5 actions without caching. How many HDFS reads?

The Problem: Repeated HDFS Reads



Quiz: You run 5 actions without caching. How many HDFS reads? **5 times!**

Why?

Spark is **lazy** — every action recomputes from scratch by replaying the DAG lineage.

Why Does Spark Re-Read? — The Library Analogy

✗ Without Caching

- Like walking to the shelf **every time** you need a fact
- Need the population of Riyadh? Walk to shelf.
- Need it again? Walk **again**.
- 5 questions = 5 trips

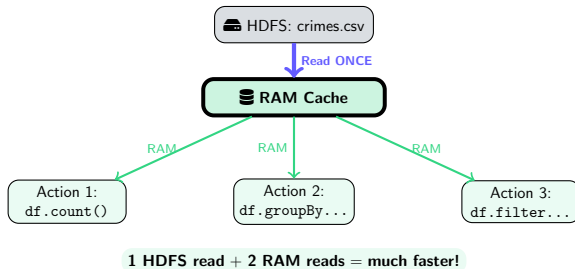
✓ With Caching

- Like **photocopying** the page
- First trip: walk + photocopy
- Every question after: read from **your desk**
- 5 questions = 1 trip + 4 instant reads

The Cost in Practice

- `df.count()` — reads 10,000 rows from HDFS
- `df.groupBy(...).show()` — reads the **same** 10,000 rows again
- With real datasets (millions of rows), each read can take **minutes**

The Solution: `.cache()` and `.persist()`



How It Works

- 1 `df.cache()` **marks** for caching (lazy!)
- 2 First action: reads HDFS, stores in RAM
- 3 Next actions: from **RAM** — 10–100× faster

Lab 07 Benchmark

Without cache: 21s + 21s + 21s = **63s**

With cache: 21s + 0.8s + 3.3s = **25s**

Speedup: **2.5×**

Caching in Code

```
# .cache() = persist in RAM only (default)
df = spark.read.csv("hdfs:///data/chicago_crimes.csv",
                    header=True, inferSchema=True)

df.cache()           # marks for caching (lazy!)
df.count()           # 1st action: reads HDFS, stores in RAM
df.groupBy("Primary Type").count().show() # from RAM!

# Unpersist when done (free memory)
df.unpersist()
```

Common Mistake

df.cache() does **NOT** cache immediately!
Caching happens on the **first action**.



Memory Tip

If DataFrame > RAM, Spark evicts partitions. Use
.persist(MEMORY_AND_DISK) instead.

Storage Levels — .persist()

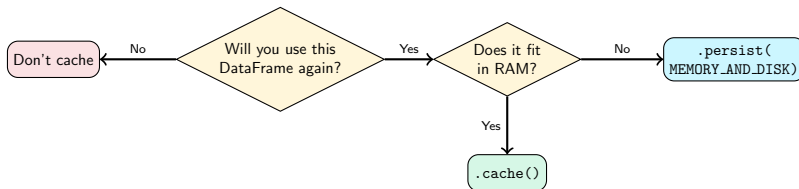
```
from pyspark import StorageLevel
df.persist(StorageLevel.MEMORY_AND_DISK)  # spill to disk
df.persist(StorageLevel.DISK_ONLY)        # disk only
```

Level	RAM	Disk	Repl.	Use Case
MEMORY_ONLY	✓	✗	No	Default (.cache()). Fastest.
MEMORY_AND_DISK	✓	✓	No	Large data that may not fit
DISK_ONLY	✗	✓	No	Very large data, save RAM
MEMORY_ONLY_2	✓	✗	2×	Critical data, fault tolerance

Rule of Thumb

Start with .cache() (MEMORY_ONLY). If you get OutOfMemory, switch to MEMORY_AND_DISK.

When to Cache — Decision Flowchart



Cache These

- DataFrames used by **multiple actions**
- **Training data** before `model.fit()`
- DataFrames after **expensive joins**

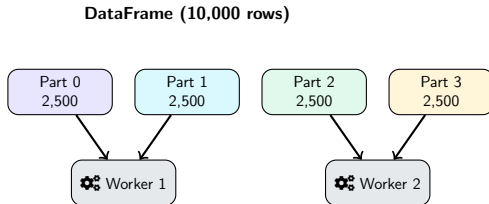
Don't Forget!

Call `df.unpersist()` when done.
Cached data stays in memory until the app ends or you free it.

Outline

- 1 Recap & The Spark Journey
- 2 Deploying Jobs with `spark-submit`
- 3 Spark Web UI Deep Dive
- 4 Caching & Persistence
- 5 Partition Tuning**
- 6 Broadcast Joins
- 7 Summary

What Are Partitions?



Analogy

4 TAs, 100 exams. Give each TA 25 = fast. Give 1 TA all 100 = slow. Partitions split data across cores the same way.

```
# Check partitions  
df.rdd.getNumPartitions()
```

Key Rule

1 partition = 1 task. Each task runs on one core of one executor.

The Goldilocks Problem — Finding the Right Partition Count

Too Few (1 partition)

1 core works
3 cores sit idle

SLOW

Just Right (8)

All 4 cores busy
2 rounds of work

FAST

Too Many (200)

Scheduling overhead
Mostly empty tasks

SLOW

The Formula

Sweet spot = $2-4 \times$ total executor cores. Our cluster: 4 cores $\rightarrow$ **8–16 partitions** is ideal.

How Partitions Are Decided

Default comes from the data source: **HDFS** $\rightarrow$ one partition per 128 MB block; **CSV** $\rightarrow$ depends on file splits. Override with `repartition(n)` or `coalesce(n)`.

repartition() vs coalesce()

repartition(n)	coalesce(n)
Full shuffle (expensive)	No shuffle (narrow dependency)
Can increase or decrease	Can only decrease
Even distribution guaranteed	Some partitions may be larger
Use for: heavy computation	Use for: writing output files

```
# Increase partitions (full shuffle)
df = df.repartition(8)

# Decrease partitions (no shuffle -- just merge)
df_out = df.coalesce(2)

# Partition by column (co-locate related rows)
df = df.repartition(4, "District")
```

Sweet Spot

Set partitions to **2–4 × total executor cores**. Our cluster: 4 cores → 8 partitions is a good default.

Shuffle Partitions — The #1 Setting to Change

After any **shuffle** (GROUP BY, JOIN, etc.), Spark creates `spark.sql.shuffle.partitions` output partitions — default is **200**.

```
# BAD for small clusters: 200 tasks on 4 cores!  
spark.conf.set("spark.sql.shuffle.partitions", 8)
```

200 partitions on 4 cores

$200 \div 4 = \mathbf{50 \text{ rounds}}$

$10,000 \text{ rows} \div 200 = 50 \text{ rows each}$

Scheduling overhead $\gg$ actual work

8 partitions on 4 cores

$8 \div 4 = \mathbf{2 \text{ rounds}}$

$10,000 \text{ rows} \div 8 = 1,250 \text{ rows each}$

Each core stays busy — efficient!

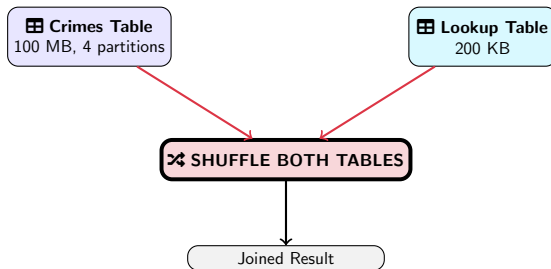
 Always Set This!

Add `--conf spark.sql.shuffle.partitions=8` to every `spark-submit` command on our cluster. This single setting can **halve** your job execution time.

Outline

- 1 Recap & The Spark Journey
- 2 Deploying Jobs with `spark-submit`
- 3 Spark Web UI Deep Dive
- 4 Caching & Persistence
- 5 Partition Tuning
- 6 Broadcast Joins**
- 7 Summary

The Shuffle Join Problem



Shuffles 100 MB + 200 KB across network = WASTEFUL

Problem

Default join shuffles **both** tables. Shuffling 100 MB of crimes data just to join with a 200 KB lookup table is extremely wasteful.

Broadcast Join — The Formula Sheet Analogy

✗ Shuffle Join

Like making **every student** line up at **one shared** formula sheet taped to the wall.

Everyone moves, everyone waits, chaos ensues. (= network shuffle)

✓ Broadcast Join

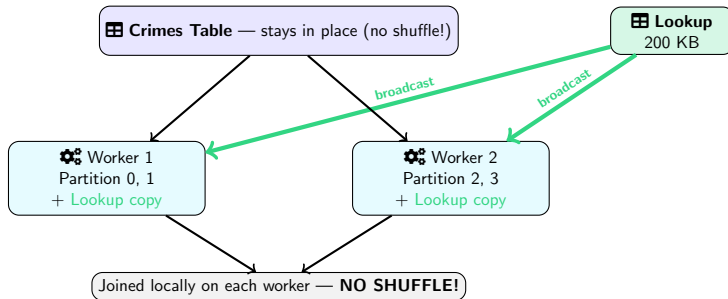
Like **photocopying** the 1-page formula sheet and giving a copy to **every student**.

Students stay seated, work in parallel. (= local join)

“A” How It Works

- The small Lookup table (200 KB) is **copied entirely** to every executor
- The large Crimes table **stays in place** — partitions don't move
- Each executor joins locally — **no shuffle!**
- Network savings: **99.6% less data** transferred

Broadcast Join — The Solution (Diagram)



Result

Small table (<10 MB) is copied to every executor. Join happens **locally** — zero shuffle of the large table!

Broadcast Join in Code

```
from pyspark.sql.functions import broadcast

# Small lookup table
district_df = spark.createDataFrame([
    (1, "Central"), (2, "Wentworth"),
    (8, "Chicago Lawn"), (11, "Harrison")
], ["DistrictID", "DistrictName"])

# WITHOUT broadcast (shuffles both tables)
result = df.join(district_df,
    df["District"] == district_df["DistrictID"])

# WITH broadcast (sends lookup to all executors)
result = df.join(broadcast(district_df),
    df["District"] == district_df["DistrictID"])
```

Auto-Broadcast

Spark auto-broadcasts tables < 10 MB

Verify with explain()

Look for BroadcastHashJoin in
result.explain()

Join Strategy Summary

Scenario	Strategy	Cost
Both tables large	Shuffle (Sort-Merge) Join	High (full shuffle)
One table small (<10 MB)	Broadcast Join	Low (no shuffle)
Same partition key	Co-partitioned join	Medium (pre-repartition)
Frequent joins on same key	Bucket join	Low (pre-bucketed)

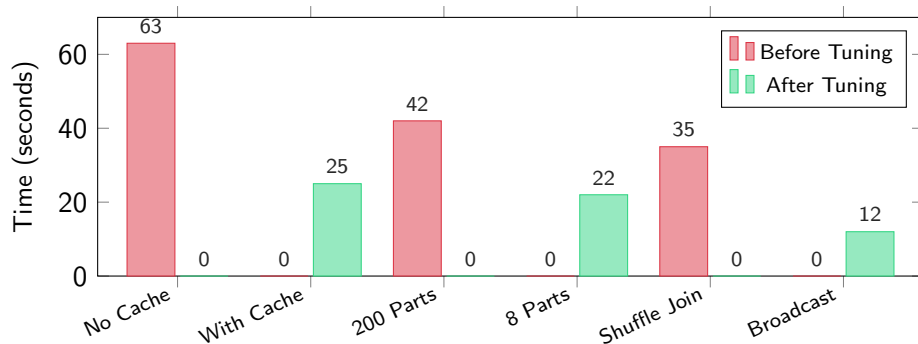
💡 Key Insight

In Big Data courses, most “lookup” joins (district names, payment types, status codes) involve a **small dimension table**. Always use `broadcast()` for these!

Outline

- 1 Recap & The Spark Journey
- 2 Deploying Jobs with `spark-submit`
- 3 Spark Web UI Deep Dive
- 4 Caching & Persistence
- 5 Partition Tuning
- 6 Broadcast Joins
- 7 Summary

Performance Benchmarks — Real Numbers



Cache: 2.5× faster — **Partitions:** 1.9× faster — **Broadcast:** 2.9× faster
Combined: all three levers together can yield 5–10× speedup on real workloads.

Key Takeaways

- ❶ **spark-submit**: deploy `.py` scripts to YARN (`client` for dev, `cluster` for production)
- ❷ **Web UI**: Jobs → Stages → Tasks. Watch shuffle bytes, straggler tasks, GC time
- ❸ **Caching**: `.cache()` keeps data in RAM; avoids repeated HDFS reads
- ❹ **Partitions**: set to $2-4 \times$ total cores. Use `repartition()` to increase, `coalesce()` to decrease
- ❺ **Broadcast joins**: send small tables to all executors; eliminates shuffle of the large table

The Three Levers of Spark Performance

Lever 1: Caching — Lever 2: Partitioning — Lever 3: Broadcast

Spark Tuning Cheat Sheet — Screenshot This!

>_ spark-submit Template:

```
spark-submit --master yarn --deploy-mode client \  
  --num-executors 2 --executor-memory 1g --executor-cores 2 \  
  --conf spark.sql.shuffle.partitions=8 my_script.py
```

Caching

Multiple actions on same DF?

→ `df.cache()`

OOM? `.persist(M&D)`

Done? `df.unpersist()`

Partitions

Set to 2–4× total cores

Our cluster: **8 partitions**

`repartition(n) / coalesce(n)`

Always set `shuffle.partitions`!

Broadcast

Small table < 10 MB?

→ `broadcast(small_df)`

Verify: `result.explain()`

Never broadcast large tables



Web UI Quick Reference

While running: `http://master:4040` — After finish:

`http://master:8088/proxy/<app-id>/`

Top 5 Student Mistakes — Avoid These!

❌ 1 Using cluster mode during development

You see no output → use `client` mode for debugging

❌ 2 Expecting `.cache()` to work immediately

It's lazy! Caching happens on the first **action** after the call

❌ 3 Leaving shuffle partitions at 200

200 tasks on 4 cores = 50 rounds of overhead. Set to 8!

❌ 4 Forgetting `spark.stop()`

Your app hogs cluster resources. Always call it at the end.

❌ 5 Broadcasting a large table

Causes `OutOfMemoryError` on every executor. Only broadcast < 10 MB.

✅ The Fix Pattern

Client mode → cache if reused → set shuffle partitions to 8 → broadcast small tables → `spark.stop()` at the end.

Putting It All Together — Optimized Script

```
# optimized_analysis.py -- applies all three tuning levers
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, count, broadcast

spark = SparkSession.builder \
    .appName("OptimizedCrimeAnalysis") \
    .config("spark.sql.shuffle.partitions", 8) \
    .getOrCreate()

df = spark.read.csv("hdfs:///data/chicago_crimes.csv",
                    header=True, inferSchema=True)
df = df.repartition(8)      # Lever 2: match core count
df.cache()                 # Lever 1: reuse across actions

df.count()                 # triggers cache
top_crimes = df.groupBy("Primary Type") \
    .agg(count("*").alias("cnt")).orderBy(col("cnt").desc())
top_crimes.show(10)        # reads from RAM!

# Lever 3: broadcast small lookup table
districts = spark.createDataFrame([(1, "Central"), (2, "Wentworth")],
                                   ["id", "name"])
joined = df.join(broadcast(districts), df["District"]==districts["id"])

df.unpersist()             # free memory
spark.stop()               # release cluster resources
```

What's Next

Week 8B: Spark MLlib — Machine Learning at Scale

- ML Pipeline API: Transformer, Estimator, Pipeline
- Feature engineering: StringIndexer, VectorAssembler
- Train a Random Forest to predict arrests in Chicago crimes
- Evaluate: AUC-ROC, accuracy, F1 score, confusion matrix

Today's Lab: Spark Performance Tuning

- Experiment 1: With vs without caching (3 actions) — target: $>2\times$ speedup
- Experiment 2: Partition count (1, 4, 8, 16) — find the sweet spot
- Experiment 3: Broadcast vs shuffle join — measure network savings
- Record timings + Web UI screenshots for each experiment

Milestone M3 Reminder

Due end of Week 8: RDD + DataFrame + MLlib analytics on Chicago Crimes.